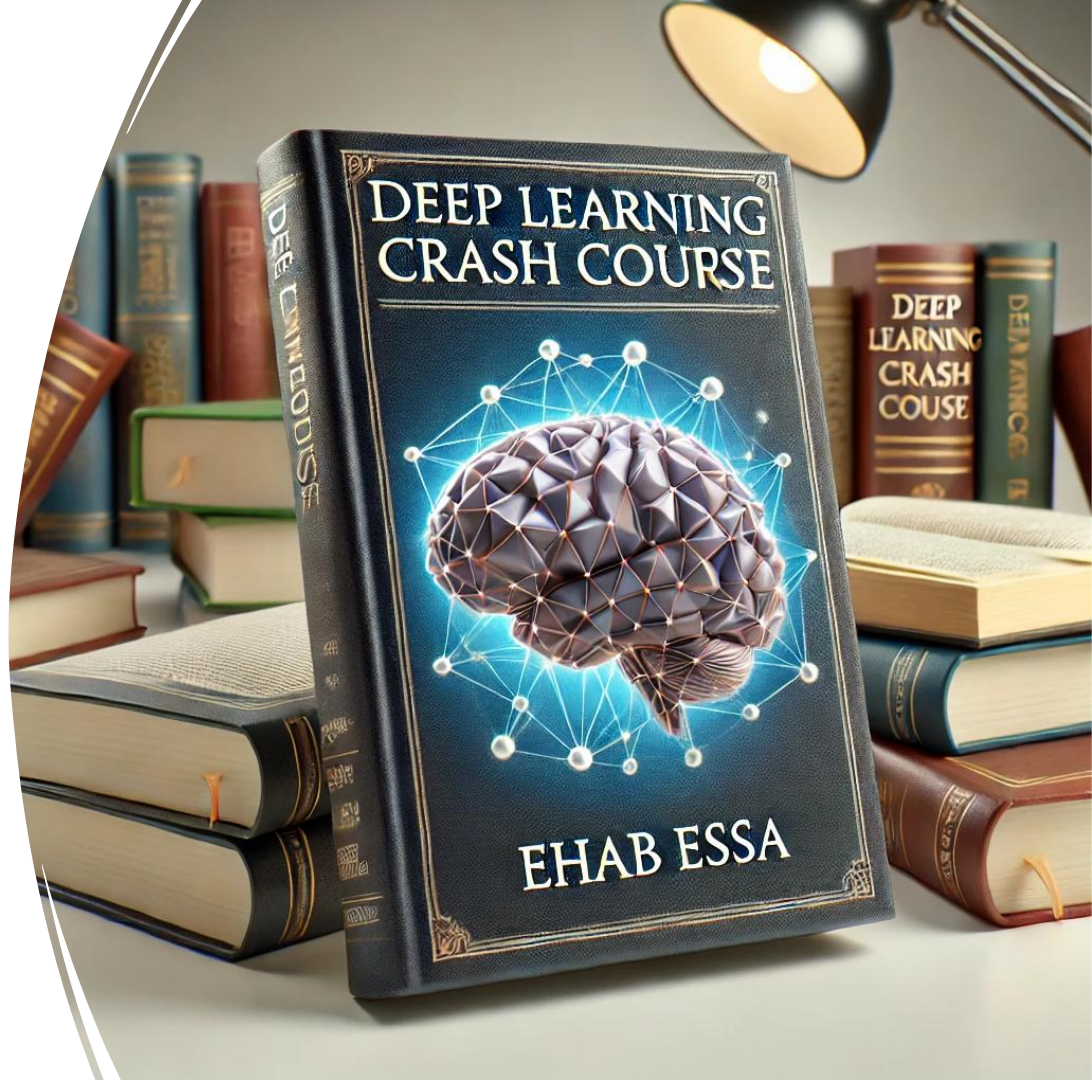




Introduction to Deep Learning

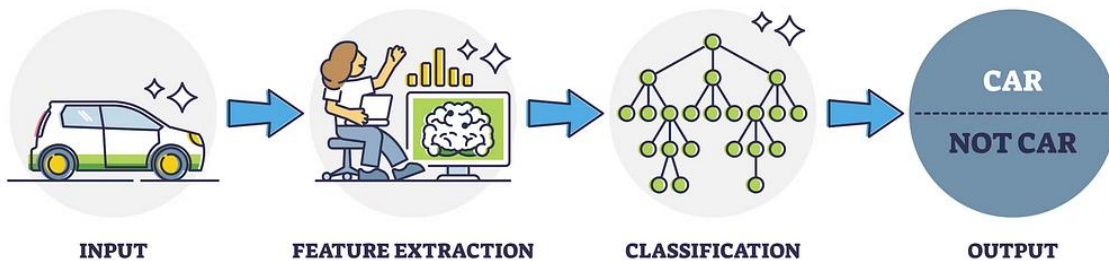
Dr. Ehab Essa

Faculty of Computers and Information,
Mansoura University, Egypt.

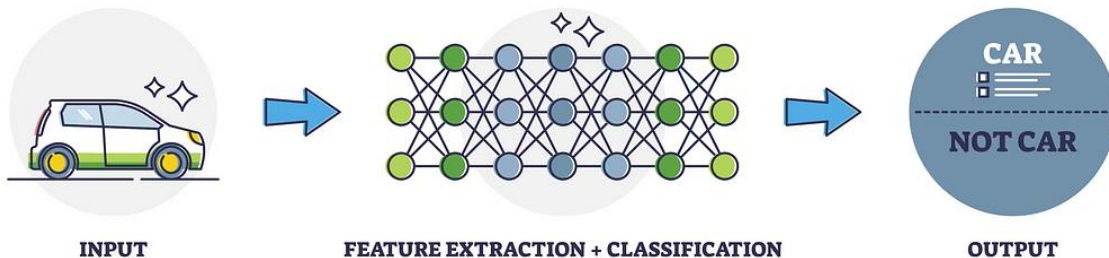


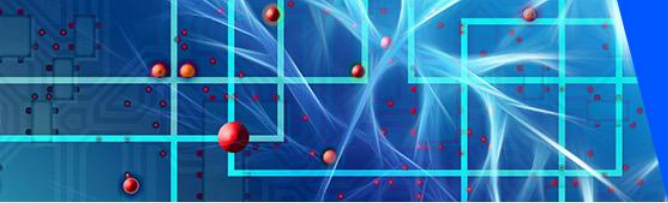
Machine learning vs Deep learning

MACHINE LEARNING



DEEP LEARNING

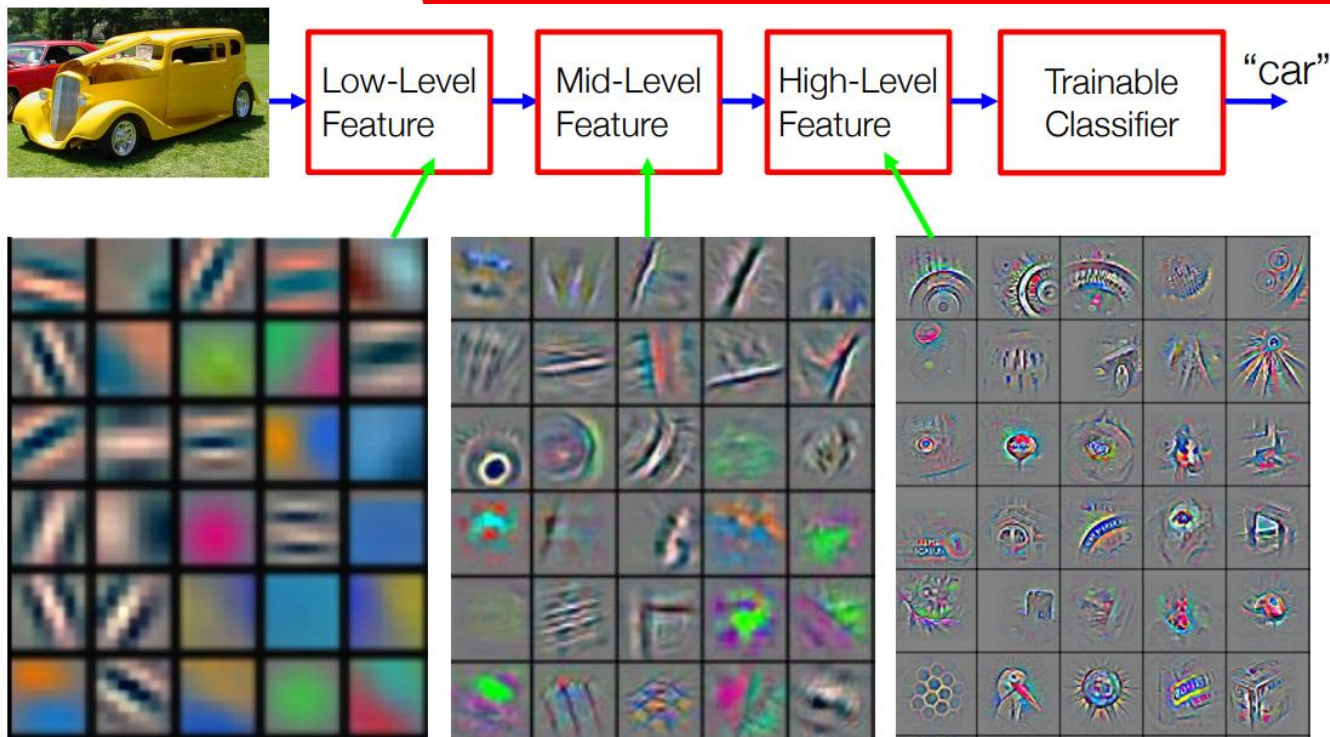




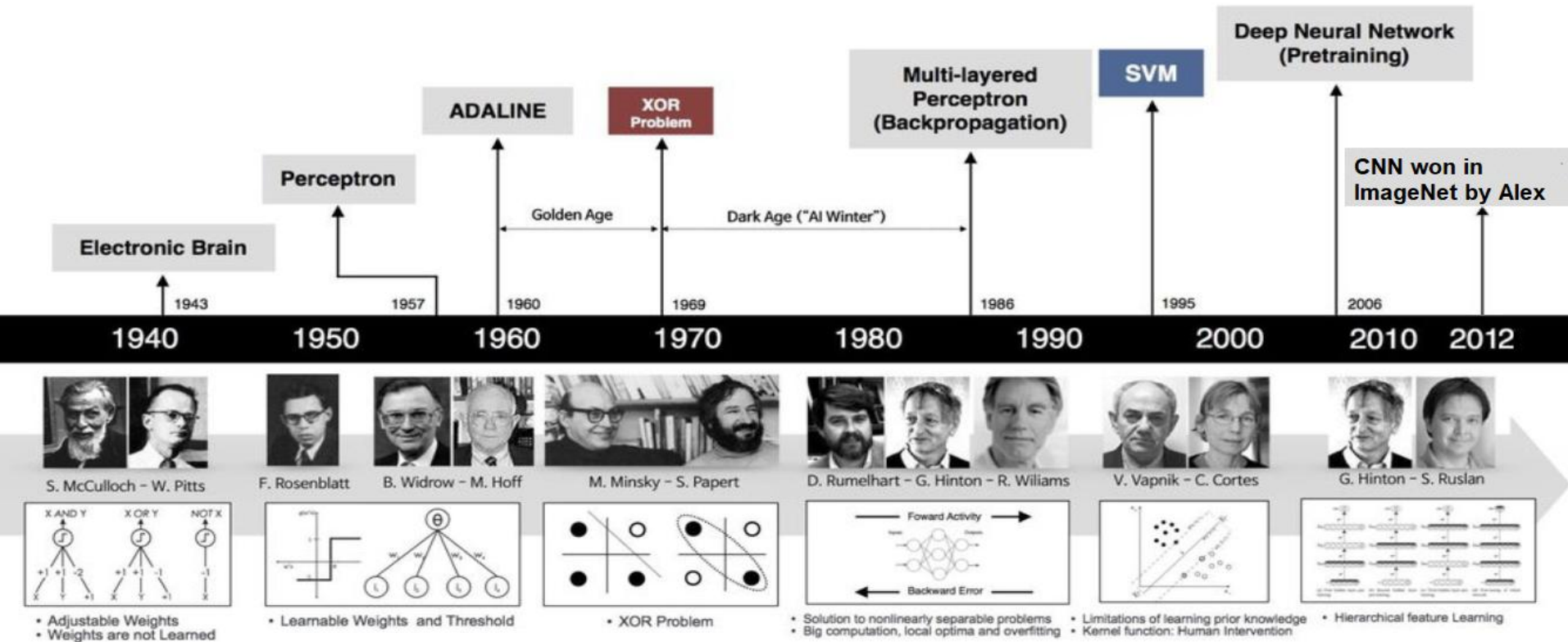
Machine learning vs Deep learning

- Machine (Shallow) learning
 - Feature extraction is a manual process that requires domain knowledge of the data that we are learning from.
 - Features are key to recent progress in recognition.
 - Trainable classifier is often generic (e.g. SVM).
- Deep learning methods have in common:
 - Learn underlying features directly from data.
 - learning of feature hierarchies representations in each layer.
 - multiple layers of nonlinear processing units.

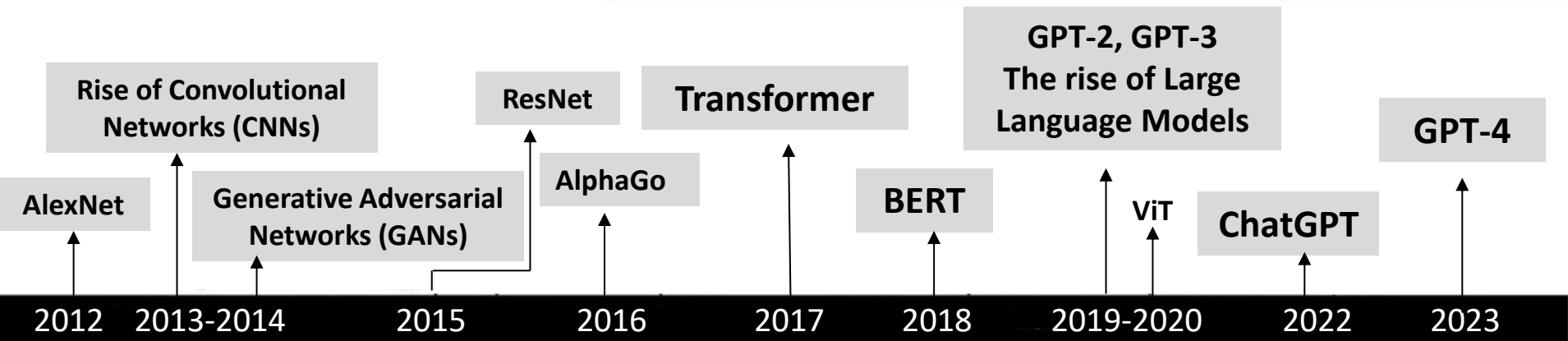
Deep Learning = Hierarchical Compositionality



Brief history of Deep Learning

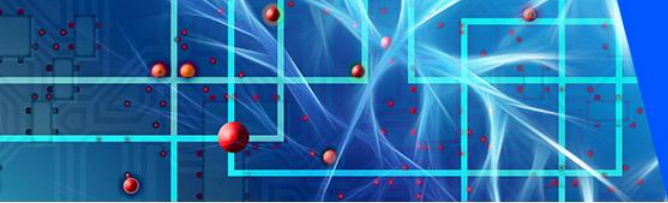


Brief history of Deep Learning



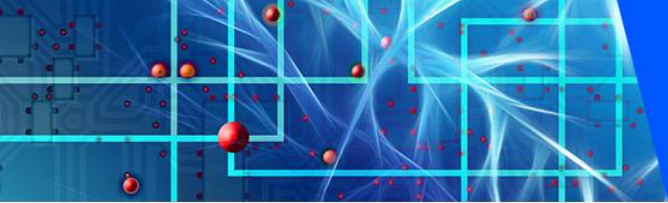
Ian
Goodfellow





Brief history of Deep Learning

- **1940s: The Birth of Artificial Neurons**
 - **1943: Warren McCulloch** and **Walter Pitts** proposed the first mathematical model of an artificial neuron. Their work laid the foundation for neural networks by mimicking the way biological neurons process information.
- **1950s: The Perceptron**
 - **1957: Frank Rosenblatt** developed the **Perceptron**, the first neural network model capable of learning from data. It was a significant breakthrough but limited in its ability to solve complex problems.
- **1960s: Early Development and Challenges**
 - **1960: Bernard Widrow** and **Marcian Hoff** introduced the **ADALINE** (Adaptive Linear Neuron) and **MADALINE** models, which were among the first to be used in practical applications like adaptive filters.
 - **1969: Marvin Minsky** and **Seymour Papert** published "*Perceptrons*," which highlighted the limitations of single-layer neural networks in solving the XOR problem, leading to a decline in neural network research—a period often referred to as the "AI Winter."
- **1980s: The Backpropagation Revolution**
 - **1986: David Rumelhart, Geoffrey Hinton, and Ronald J. Williams** rediscovered and popularized the **backpropagation** algorithm, which allowed multi-layered networks (often called deep networks) to be trained effectively. This rekindled interest in neural networks.
- **1990s: SVMs and Renewed Interest**
 - **1995: Vladimir Vapnik** and **Corinna Cortes** introduced **Support Vector Machines (SVMs)**, which became popular due to their effectiveness in classification tasks. Neural networks faced competition from SVMs and other machine learning methods during this time.
- **2000s: The Deep Learning Era Begins**
 - **2006: Geoffrey Hinton** and his colleagues introduced **pretraining** techniques using deep belief networks, which made training deep neural networks more feasible and effective. This marked the beginning of the deep learning era.



Brief history of Deep Learning

- **2010s: Breakthroughs in Deep Learning**
 - **2012: AlexNet**, developed by **Alex Krizhevsky**, **Ilya Sutskever**, and **Geoffrey Hinton**, won the ImageNet competition by a large margin, showcasing the power of Convolutional Neural Networks (CNNs) and sparking widespread interest in deep learning.
- 2014: Generative Adversarial Networks (GANs), introduced by Ian Goodfellow, revolutionized generative modeling, enabling the creation of realistic images, videos, and more.
- 2015: Residual Networks (ResNet) by He et al. introduced the concept of skip connections, enabling the training of ultra-deep networks (over 100 layers) without the vanishing gradient problem.
- 2016: AlphaGo: Google DeepMind's AlphaGo defeated the world champion Go player, marking a milestone in deep reinforcement learning. AlphaGo combined deep neural networks with reinforcement learning
- 2017: **Transformers**, introduced by Vaswani et al. Google Brain, revolutionized natural language processing (NLP), leading to models like BERT and GPT that have set new benchmarks in various NLP tasks. Unlike RNNs, Transformers use self-attention mechanisms to process input sequences in parallel, enabling faster and more effective training.
- 2018 **BERT** (Bidirectional Encoder Representations from Transformers), introduced by Google, transformed NLP tasks by applying pre-training followed by fine-tuning on specific tasks. Use Transfer learning.
- **2020: The Rise of Large-Scale Models**
 - OpenAI introduced GPT-2, a large-scale language model trained to generate coherent and fluent text, pushing the boundaries of natural language understanding and generation. This sparked widespread interest in large-scale language models, showcasing the power of unsupervised learning on massive datasets.
 - OpenAI's GPT-3 (175 billion parameters) became the largest and most powerful language model at the time, demonstrating unprecedented capabilities in text generation, translation, summarization, and even code writing.
 - GPT-3.5 was introduced in late 2022 as an upgraded version of GPT-3, with enhanced conversational capabilities. This version powered the initial ChatGPT product,
 - ChatGPT is a fine-tuned version of GPT-3, specifically optimized for conversation.
 - Training: ChatGPT was fine-tuned using Reinforcement Learning from Human Feedback (RLHF). Human AI trainers provided conversations and rated responses, which allowed the model to learn not just from static data but from interactions and preferences.
 - Multimodal capabilities: GPT-4 introduced multimodal abilities. March 14, 2023. This means it can accept both text and images as inputs, making it capable of tasks such as image captioning, understanding complex visual scenes, and integrating text with images for more contextual understanding.

Why Now?

- Neural Networks date back decades, so why the dominance?

1. Big Data

- Larger Datasets
- Easier Collection & Storage

IMAGENET



2. Hardware

- Graphics Processing Units (GPUs)
- Massively Parallelizable



3. Software

- Improved Techniques
- New Models
- Toolboxes



ImageNet Challenge

IMAGENET



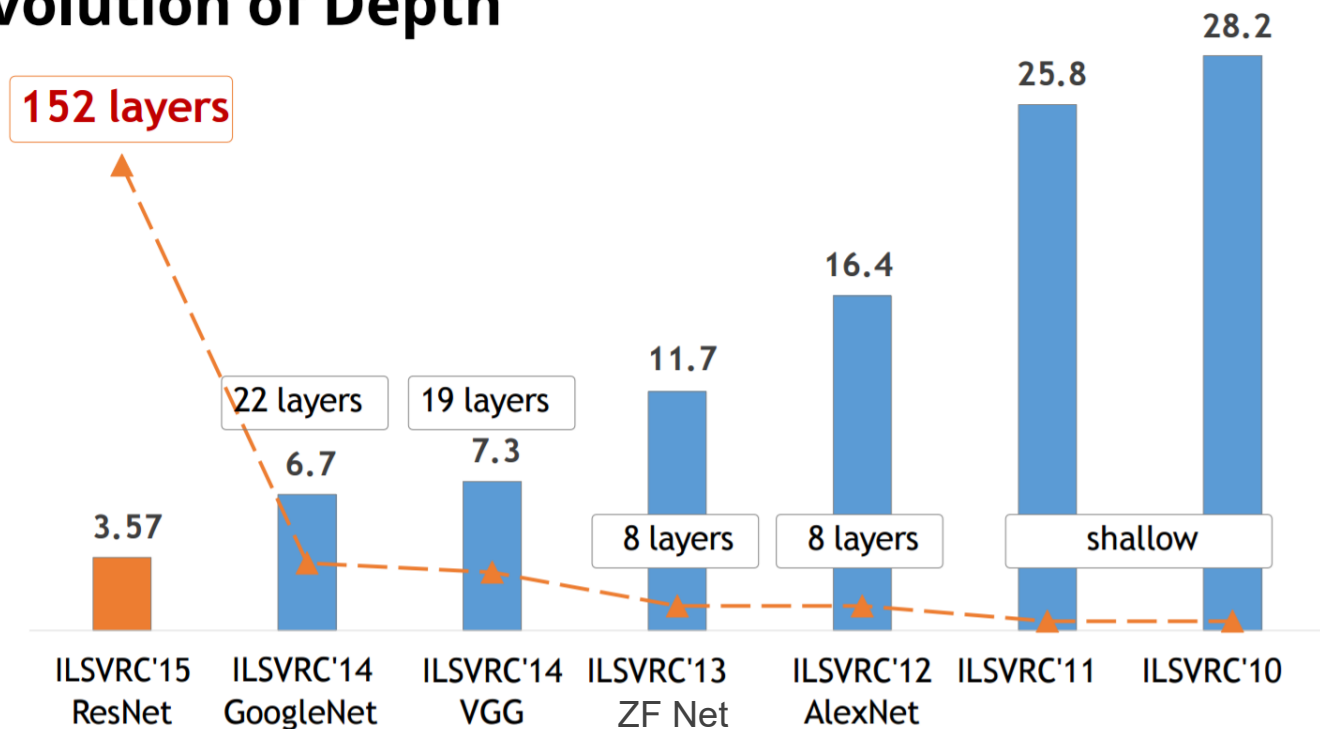
[Deng et al. CVPR 2009]

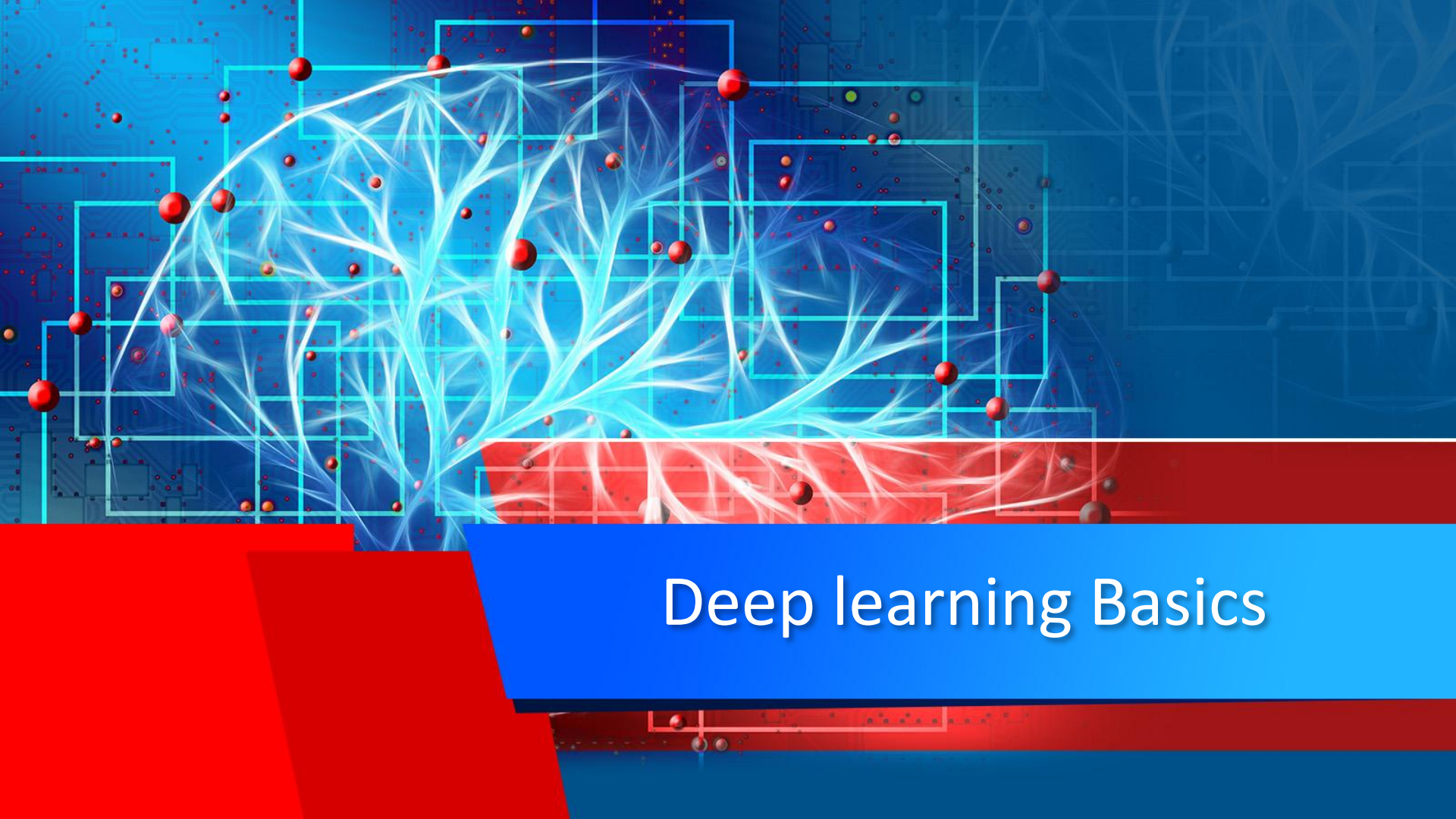
**ImageNet Large Scale Visual Recognition
Challenge (ILSVRC)**

- ~14 million labeled images, 20k classes
- Images gathered from Internet
- Human labels via Amazon Turk
- Challenge: 1.2 million training in 1000 classes
- Classification task: produce a list of object categories present in image. 1000 categories.

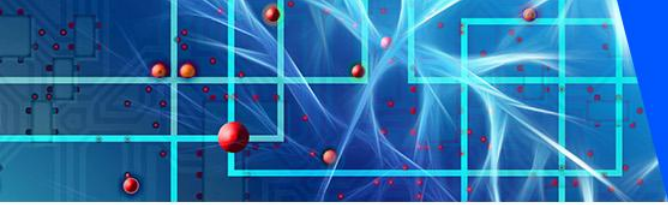
ImageNet Challenge & Deep learning

Revolution of Depth





Deep learning Basics

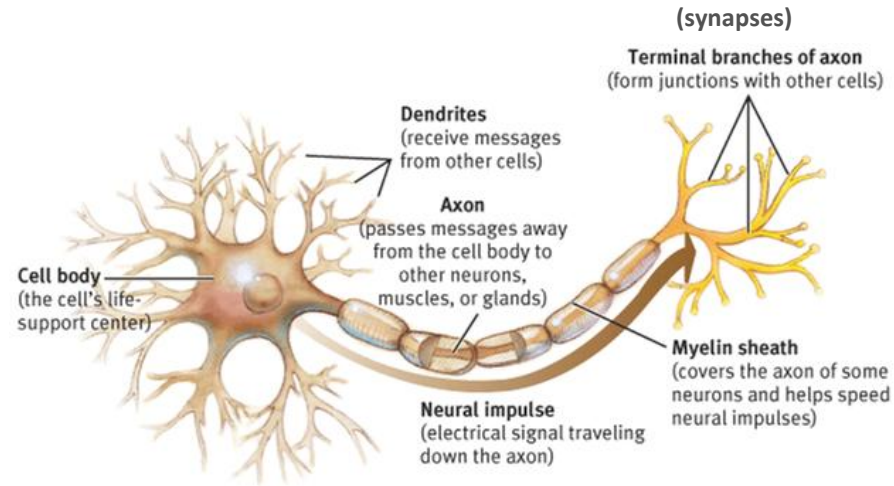


Neural Network Definition

- **Artificial neural network** is a computational model that mimics the way the human brain operates.
- Based on the assumptions :
 - Information processing occurs at many simple elements called **neurons**.
 - Signals are passed between neurons over **connection** links.
 - Each connection link has an associated **weight**, which, in a typical neural net, multiplies the signal transmitted.
 - Each neuron applies an **activation function** (usually nonlinear) to its net input (sum of weighted input signals) to determine its output signal.

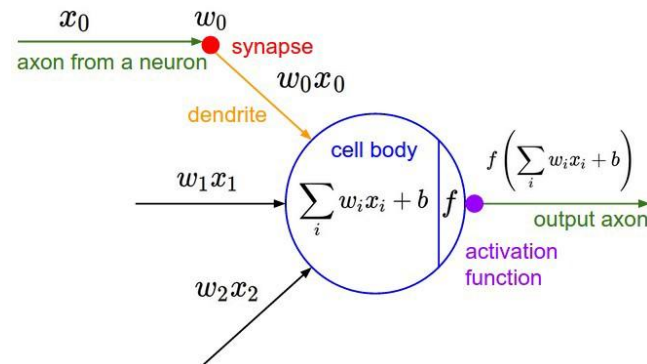
Biological Neurons

- We estimate around 10^{10} and 10^{11} the number of neurons in the human brain:
 - they receive information from other neurons through their **dendrites**
 - the “process” the information in their **cell body** (soma)
 - they send information through a “cable” called an **axon**
 - the point of connection between the axon branches and other neurons’ dendrites are called **synapses**, which regulate a chemical connection whose strength affects the input to the cell.

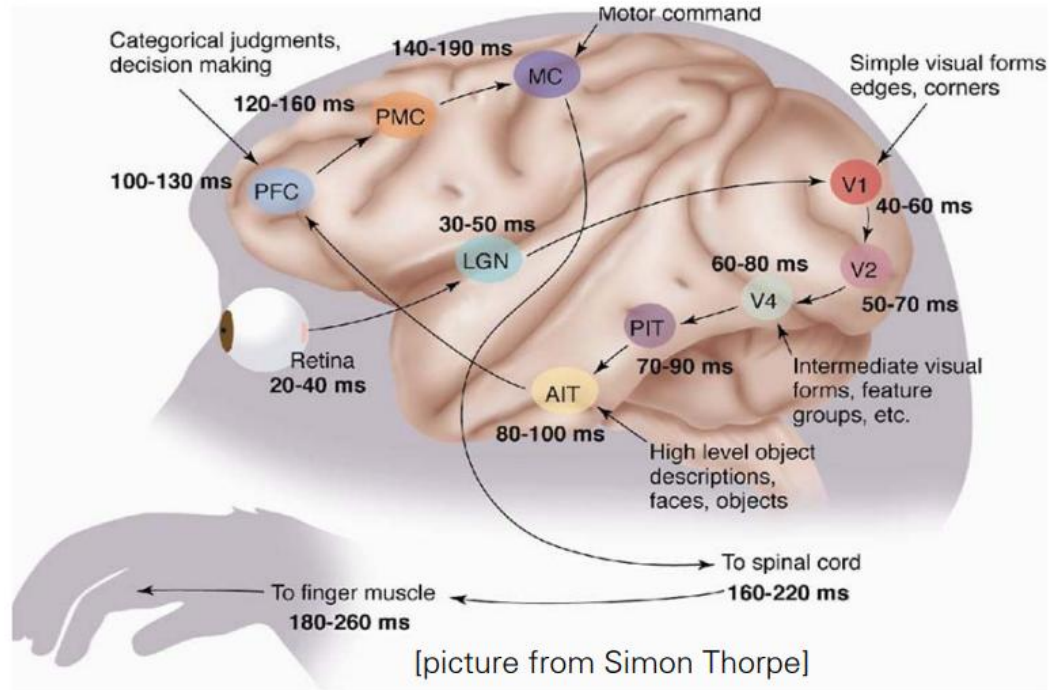


Biological Neurons

- This is what artificial neurons approximate:
 - **Weights (synapse)** between neurons model whether neurons excite or inhibit each other
 - **Inputs (dendrites)** that carry the signal to neuron (cell body) where they all get summed
 - **Summation of inputs and activation model** processes the incoming activations and converts them into output activations (**cell body**)



Visual Cortex



Artificial Neuron

- An **artificial neuron** is the basic building block that construct complicated neural networks which will make a particular computation based on other units it's connected to.

$$a(\mathbf{x}) = b + \sum_i w_i x_i = b + \mathbf{w}^\top \mathbf{x}$$

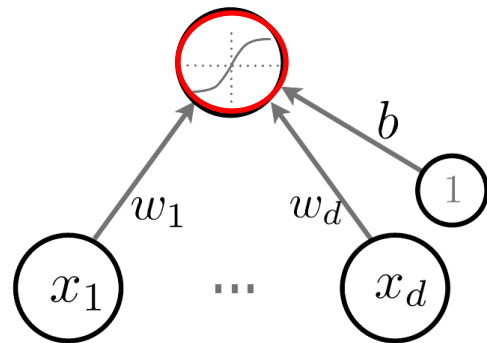
- Neuron (output) activation

$$h(\mathbf{x}) = g(a(\mathbf{x})) = g(b + \sum_i w_i x_i)$$

$\mathbf{w}$ are the connection weights

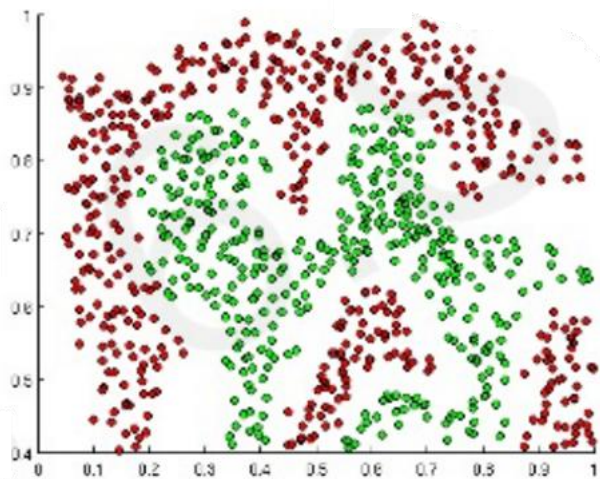
b is the neuron bias

$g(\cdot)$ is called the activation function



Importance of Activation Functions

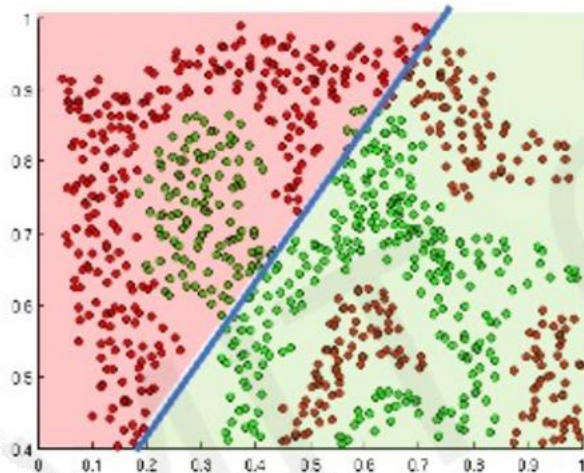
- The purpose of activation functions is to introduce **non-linearities** into the network.



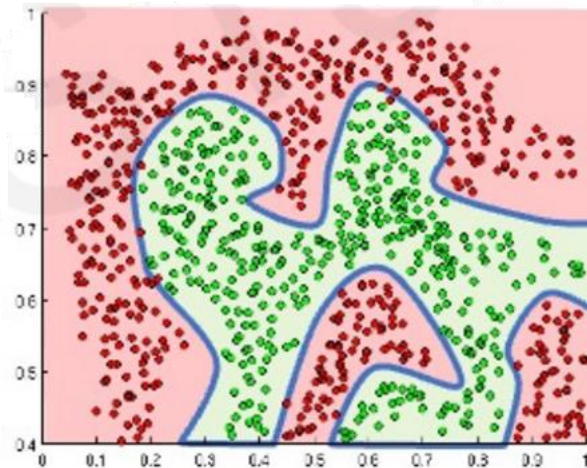
What if we wanted to build a neural network to distinguish green vs red points?

Importance of Activation Functions

- The purpose of activation functions is to introduce **non-linearities** into the network.



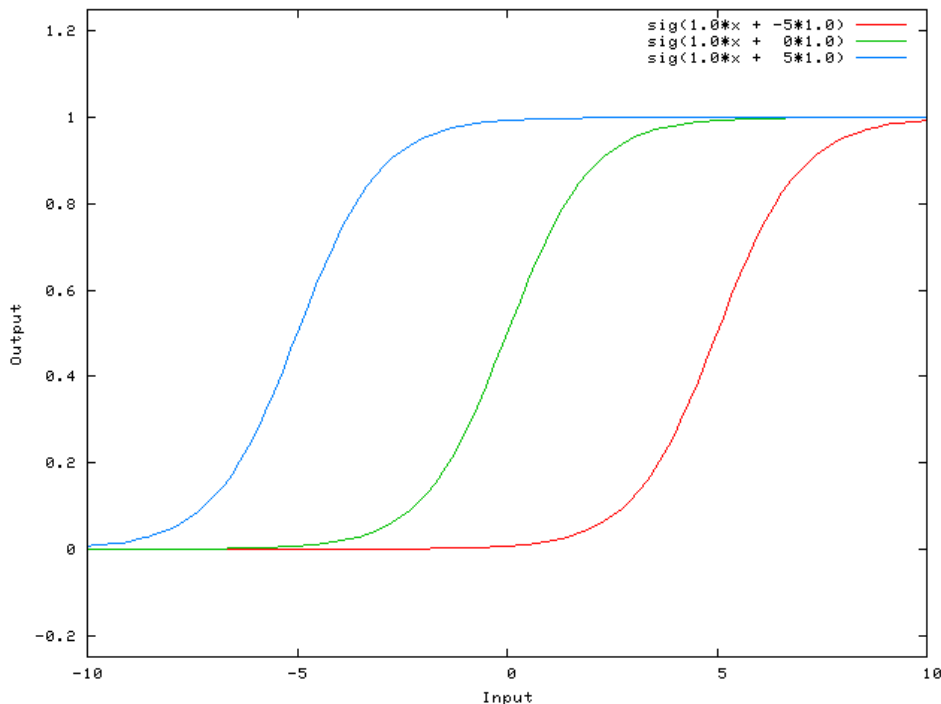
Linear activation functions produce linear decisions no matter the network size

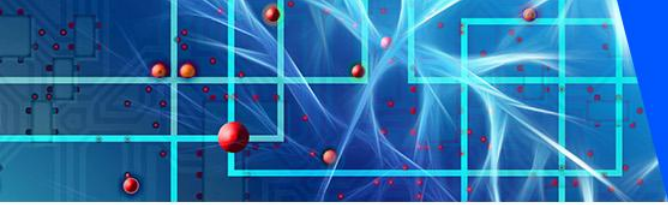


Non-linearities allow us to approximate arbitrarily complex functions

Artificial Neuron

- Bias unit
 - allows the activation function to be shifted to the left or right, to better fit the data.
 - the bias only influences the output values, it doesn't interact with the actual input data.
- While changes to the weights alter the steepness of the activation function.

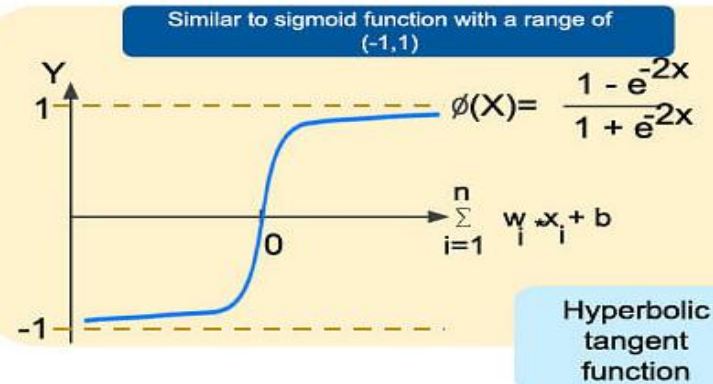
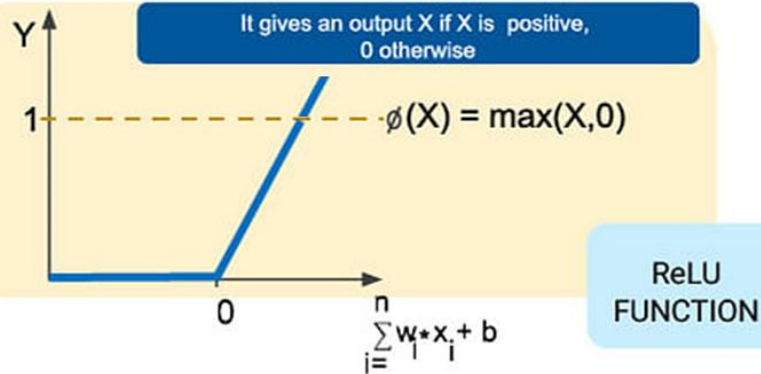
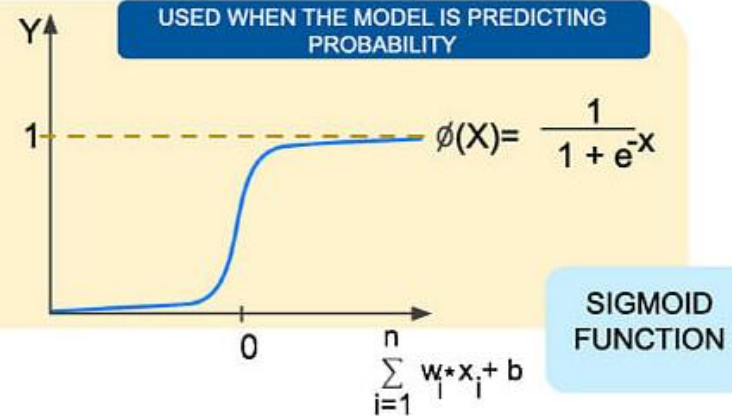
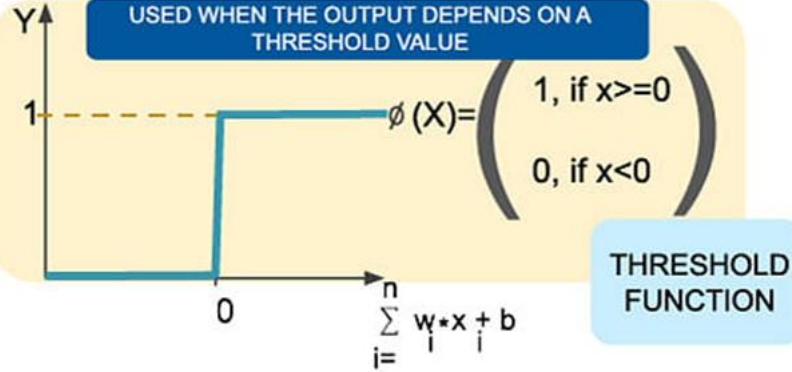




Neural Network Characterization

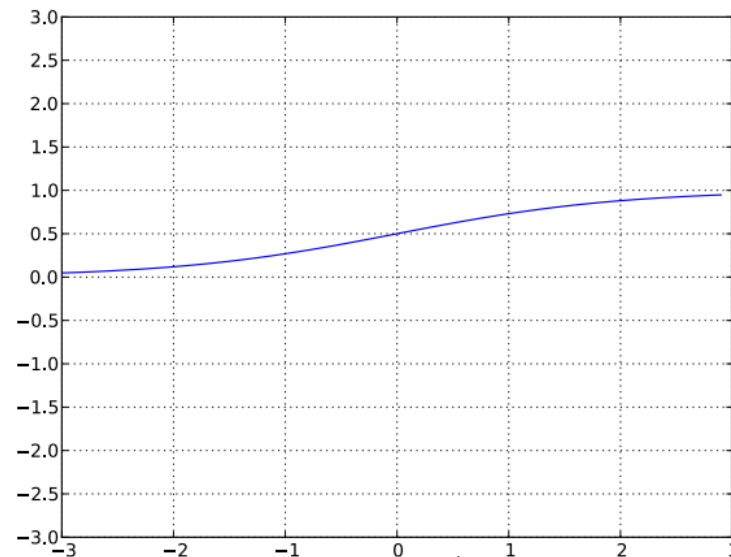
- A neural network is characterized by
 - its pattern of connections between the neurons (called its **architecture**),
 - its **activation function** , and
 - its method of determining the weights on the connections (called its **training**, or **learning**, **algorithm**)
 - This Includes determining many **hyperparameters** such as learning rate, epochs, and batch size.

Activation Functions



Activation Functions

- **Sigmoid activation function**
 - Squashes the neuron's pre-activation between 0 and 1
 - Good for probabilistic interpretations
 - Bounded
 - Strictly increasing
- Disadvantages
 - **Vanishing gradient**—for very high or very low values of a , there is almost no change to the prediction, causing a vanishing gradient problem.
 - Computationally expensive

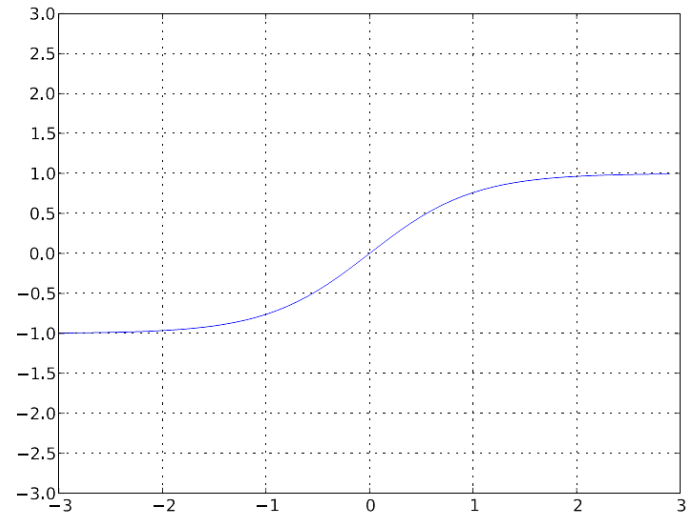


$$g(a) = \frac{1}{1 + \exp(-a)}$$

$$g'(a) = g(a)(1 - g(a))$$

Activation Functions

- **Hyperbolic tangent** (“tanh”) activation function
 - Squashes the neuron’s pre-activation between -1 and 1
 - Zero-centered output, often leads to faster convergence than Sigmoid.
 - Bounded, Strictly increasing
 - Also saturates, causing vanishing gradients.

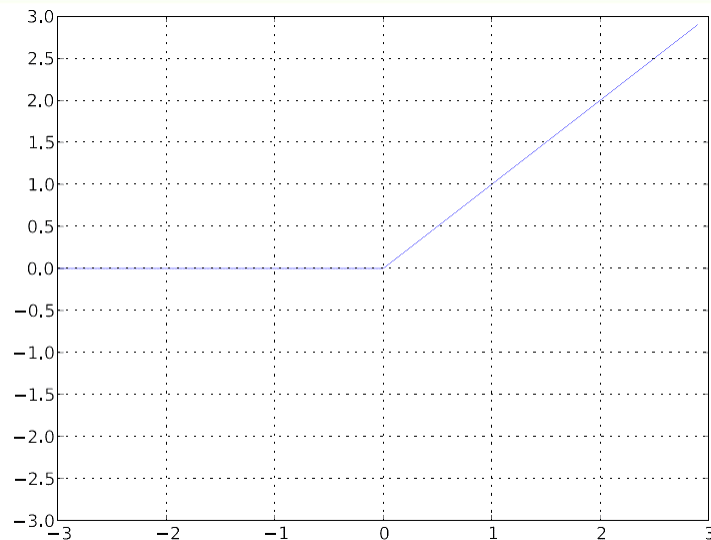


$$g(a) = \tanh(a) = \frac{\exp(a) - \exp(-a)}{\exp(a) + \exp(-a)} = \frac{\exp(2a) - 1}{\exp(2a) + 1}$$

$$g'(a) = (1 - g(a)^2)$$

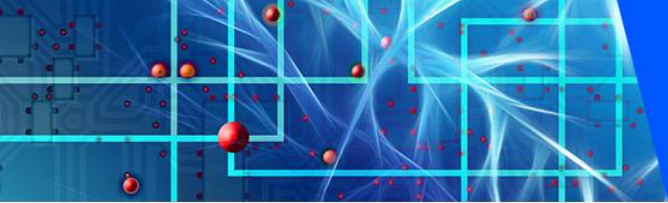
Activation Functions

- **Rectified linear (ReLU) activation function**
 - Bounded below by 0 (always non-negative), Not upper bounded
 - Tends to give neurons with sparse activities
 - Non-saturating for $x > 0$, reducing vanishing gradient problems and often speeding up training.
 - Computationally efficient—allows the network to converge very quickly
- **The Dying ReLU problem**—when inputs approach zero, or are negative, the gradient of the function becomes zero, the network cannot perform backpropagation and cannot learn.
 - Leaky ReLU, Parametric ReLU



$$g(a) = \text{reclin}(a) = \max(0, a)$$

$$g'(a) = 1, \text{ if } a > 0$$



Softmax Activation Function

- For multi-class classification:
 - we need multiple outputs (1 output per class)
 - we would like to estimate the conditional probability $p(y = c|\mathbf{x})$

$$\mathbf{o}(\mathbf{a}) = \text{softmax}(\mathbf{a}) = \left[\frac{\exp(a_1)}{\sum_c \exp(a_c)} \cdots \frac{\exp(a_C)}{\sum_c \exp(a_c)} \right]^\top$$

Vector of output neurons, where a_c is pre-activation of neuron c .

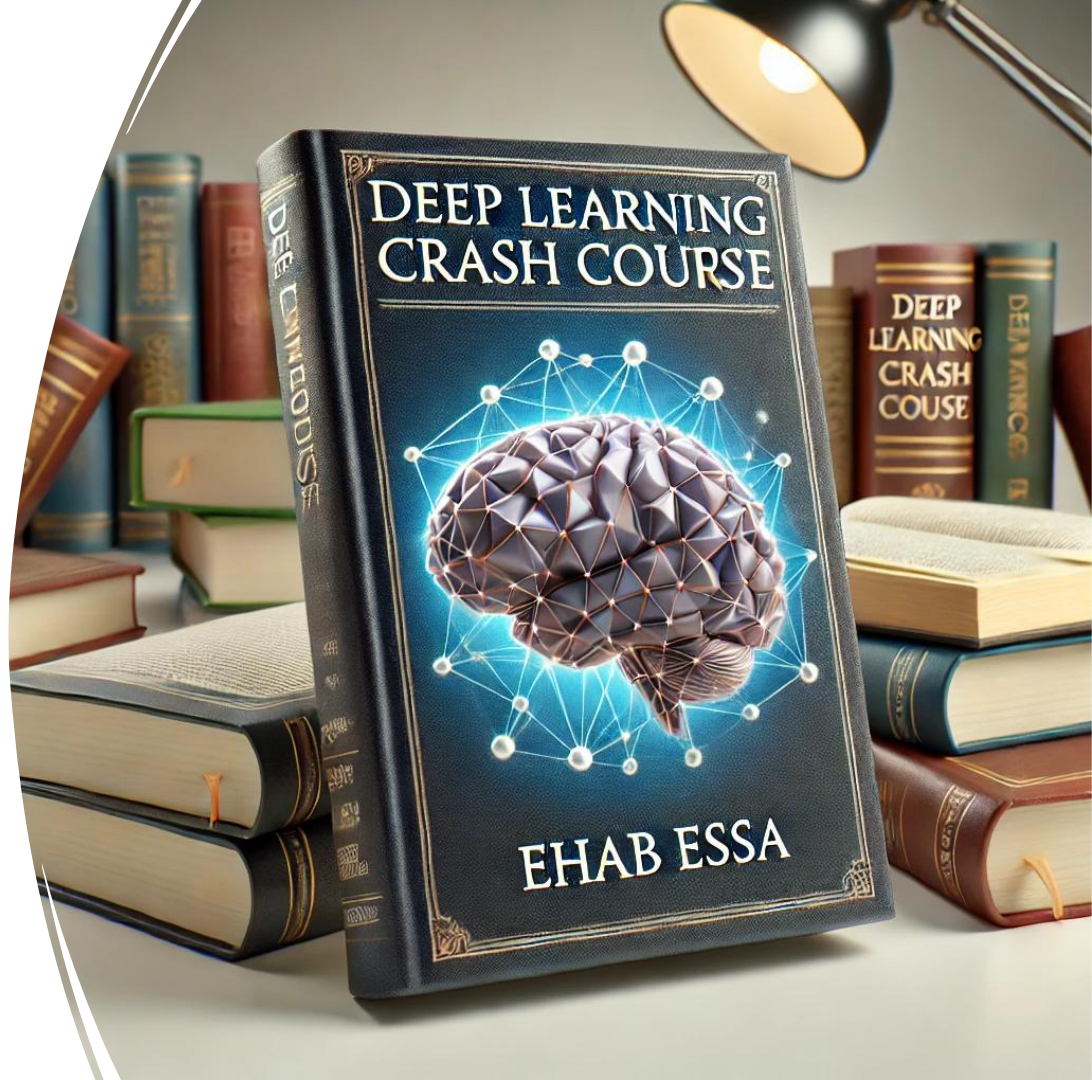
- We use the softmax activation function at the output:
 - strictly positive
 - sums to one
- Predicted class is the one with highest estimated probability



Deep Learning Architectures

Dr. Ehab Essa

Faculty of Computers and Information, Mansoura
University, Egypt.



Deep Learning Architectures

A mostly complete chart of Neural Networks

©2019 Fjodor van Veen & Stefan Leijnen asimovinstitute.org

- Input Cell
- Backfed Input Cell
- △ Noisy Input Cell
- Hidden Cell
- Probabilistic Hidden Cell
- △ Spiking Hidden Cell
- Capsule Cell
- Output Cell
- Match Input Output Cell
- Recurrent Cell
- Memory Cell
- △ Gated Memory Cell
- Kernel
- Convolution or Pool

Perceptron (P)



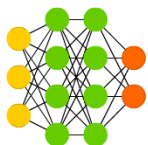
Feed Forward (FF)



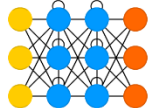
Radial Basis Network (RBF)



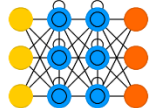
Deep Feed Forward (DFF)



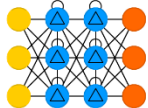
Recurrent Neural Network (RNN)



Long / Short Term Memory (LSTM)



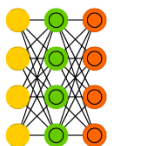
Gated Recurrent Unit (GRU)



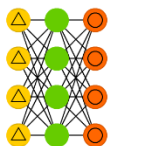
Auto Encoder (AE)



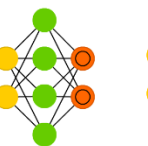
Variational AE (VAE)



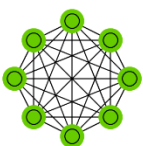
Denoising AE (DAE)



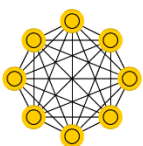
Sparse AE (SAE)



Markov Chain (MC)



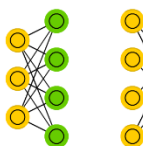
Hopfield Network (HN)



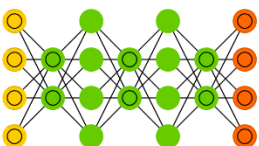
Boltzmann Machine (BM)



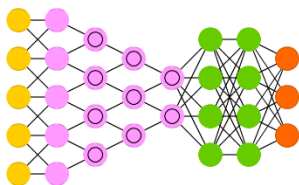
Restricted BM (RBM)



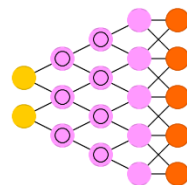
Deep Belief Network (DBN)



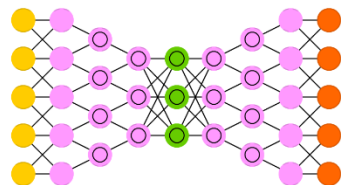
Deep Convolutional Network (DCN)



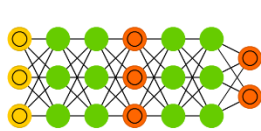
Deconvolutional Network (DN)



Deep Convolutional Inverse Graphics Network (DCIGN)



Generative Adversarial Network (GAN)



Liquid State Machine (LSM)



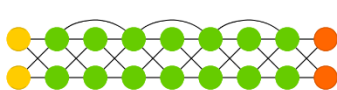
Extreme Learning Machine (ELM)



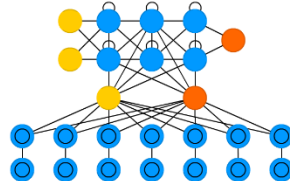
Echo State Network (ESN)



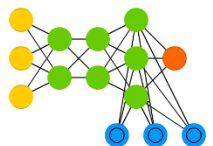
Deep Residual Network (DRN)



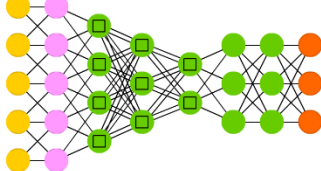
Differentiable Neural Computer (DNC)



Neural Turing Machine (NTM)



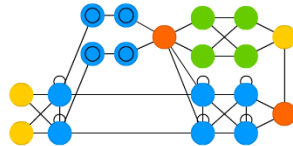
Capsule Network (CN)



Kohonen Network (KN)

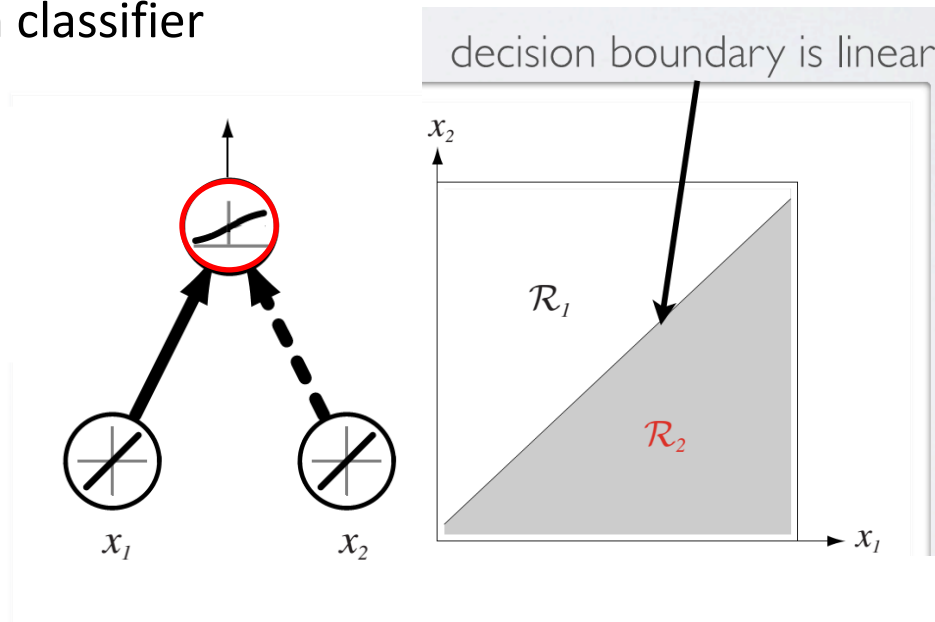


Attention Network (AN)



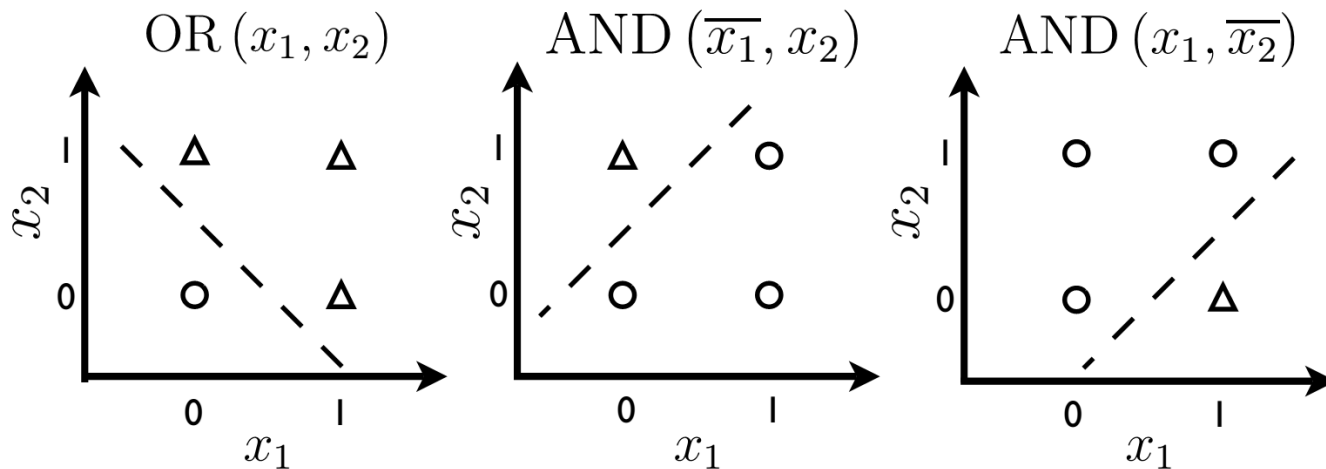
Capacity of Single Neuron

- Could do binary classification:
 - with sigmoid, can interpret neuron as estimating $p(y = 1 | \mathbf{x})$
 - also known as logistic regression classifier
 - if greater than 0.5, predict class 1
 - otherwise, predict class 0



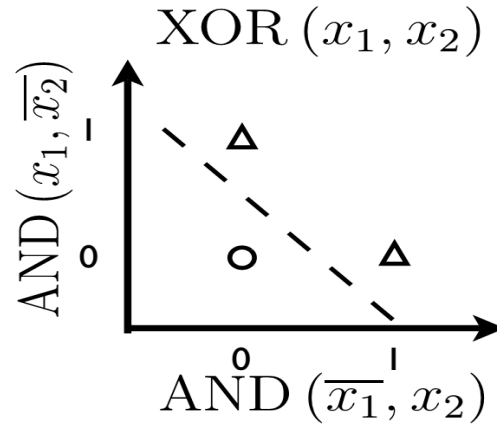
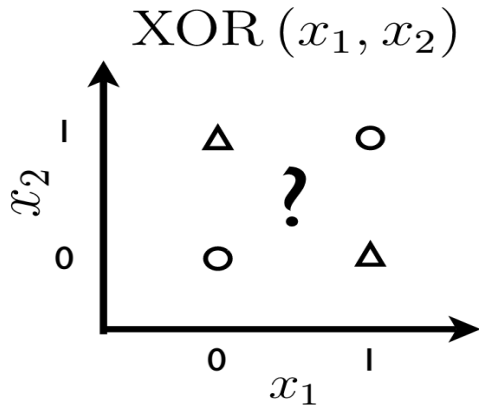
Capacity of Single Neuron

- Can solve linearly separable problems



Capacity of Single Neuron

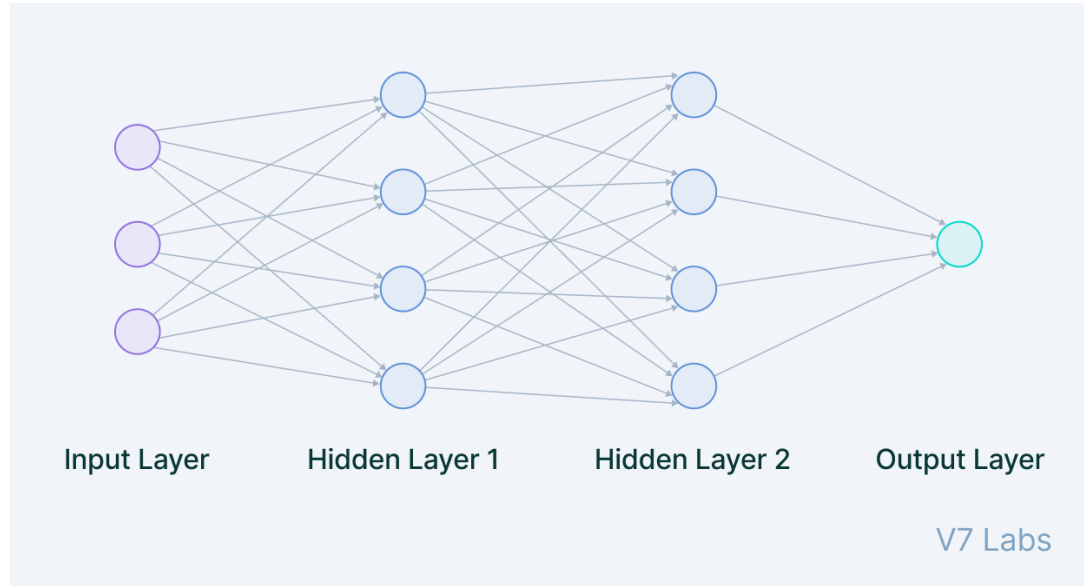
- Can't solve non linearly separable problems...



- ... unless the input is transformed in a better representation

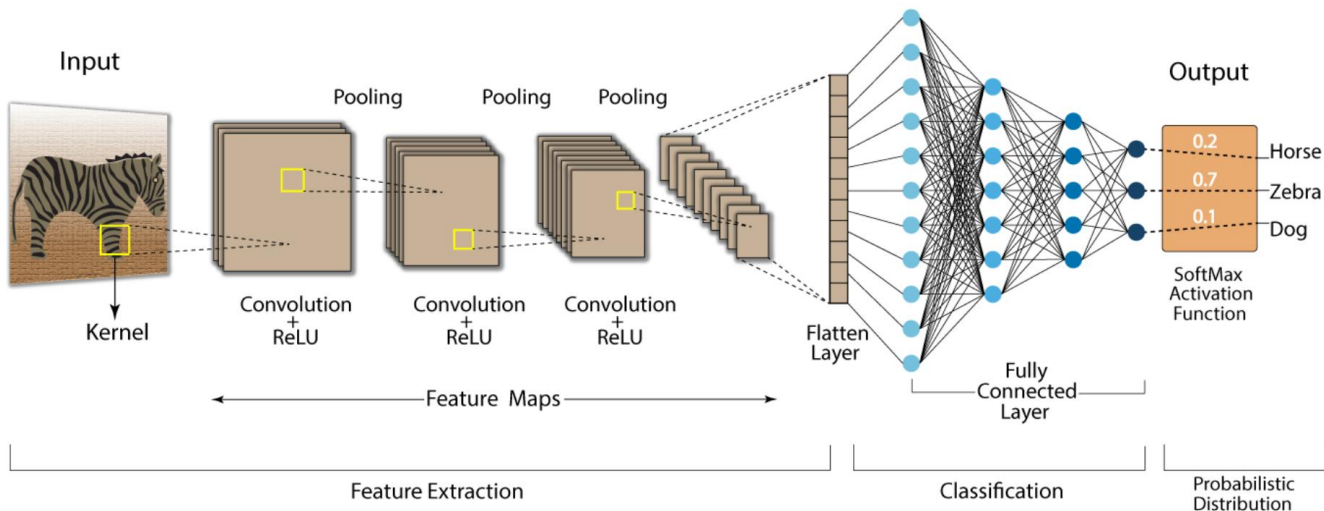
Multi-layer Neural Network

- This is a feedforward neural Network.
- The input units are fully connected to output units.
- Has one or more hidden layers of nodes.
- Can solve more complicated problems than single-layer nets, but training may be more difficult.



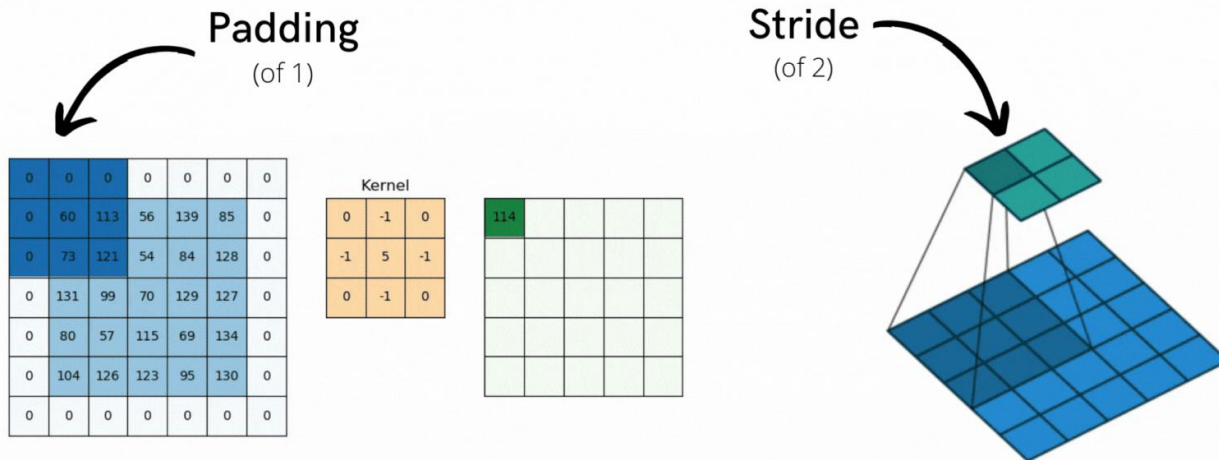
Convolutional Neural Networks (CNNs)

- CNN is a multilayer feedforward that use minimal preprocessing. This means that the network learns features that in traditional algorithms were hand-engineered.
- Their wide applications is in image and video recognition, and natural language processing.
- It has the ability to automatically and adaptively learn spatial hierarchies of features from input images.



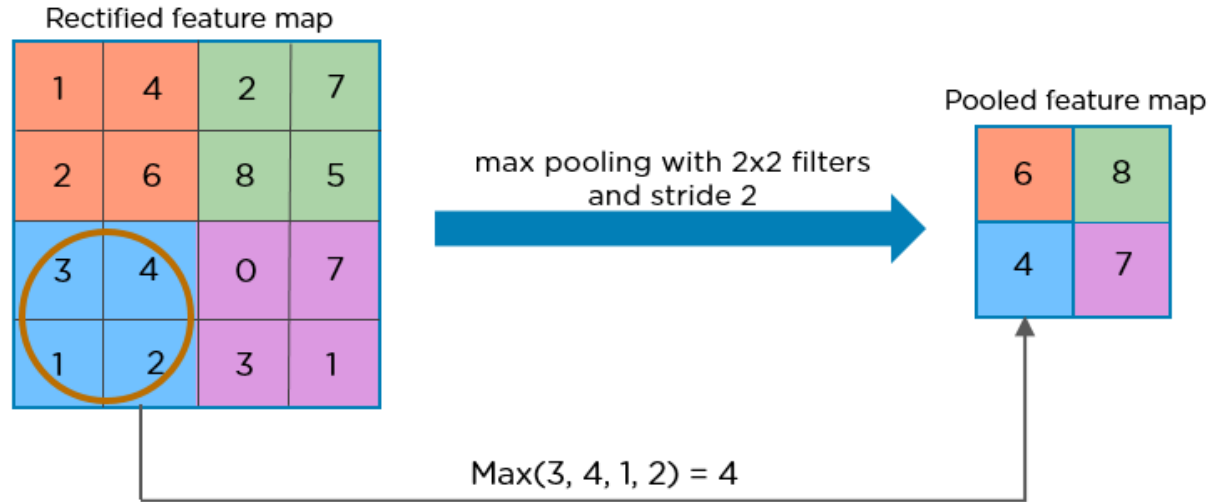
Convolution layer

- A convolution layer has **several filters** that perform the convolution operation.
- It performs convolution operations as it is scanning the input I with respect to its dimensions.
- Its hyperparameters include the **filter size F , stride S , and Padding P** .



Pooling layer

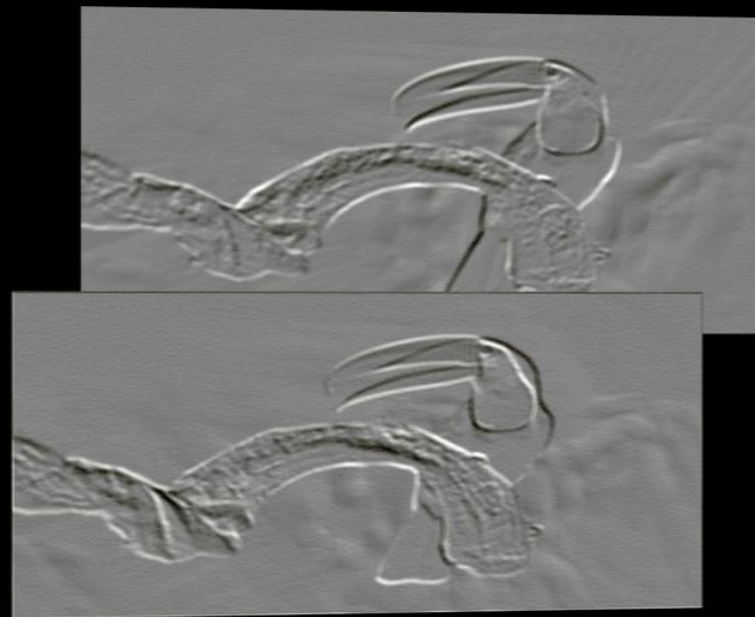
- This layer is a down-sampling operation that reduces the dimensionality of the feature maps while retaining the most essential information.
- Common types include max pooling and average pooling.



Example



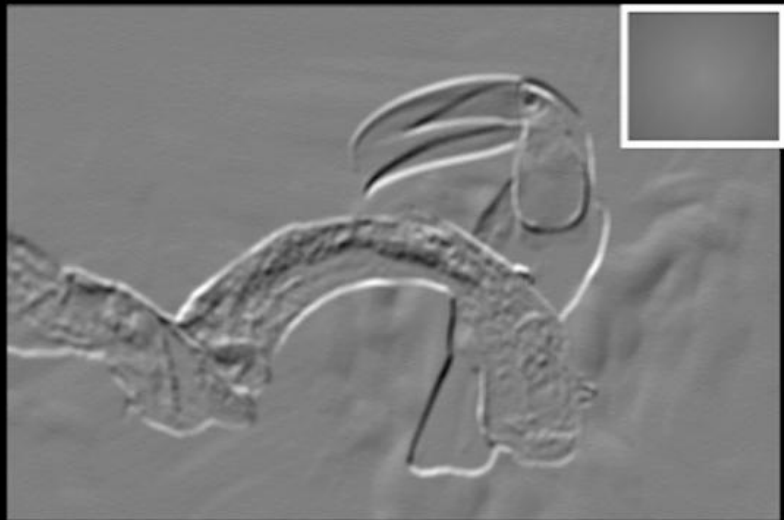
Input



Feature Map

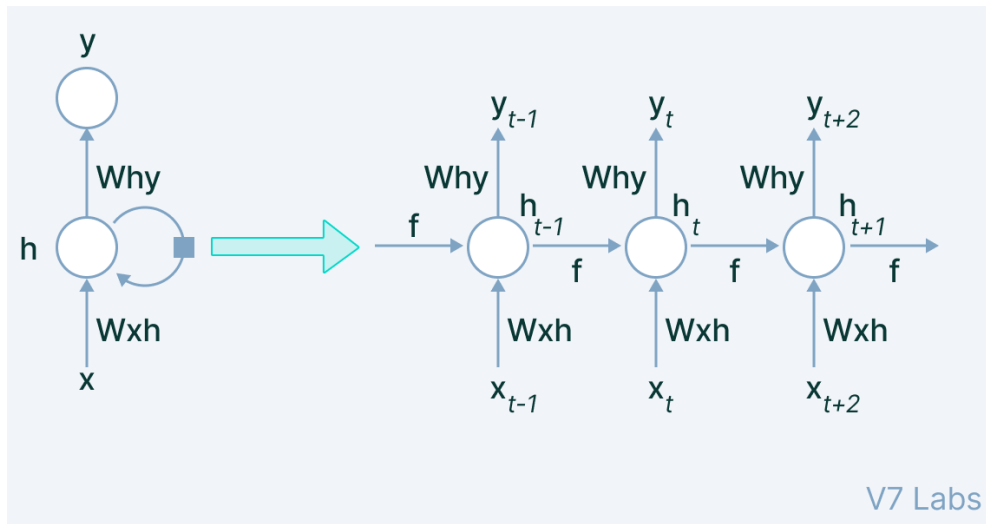
Example

Input Feature Map



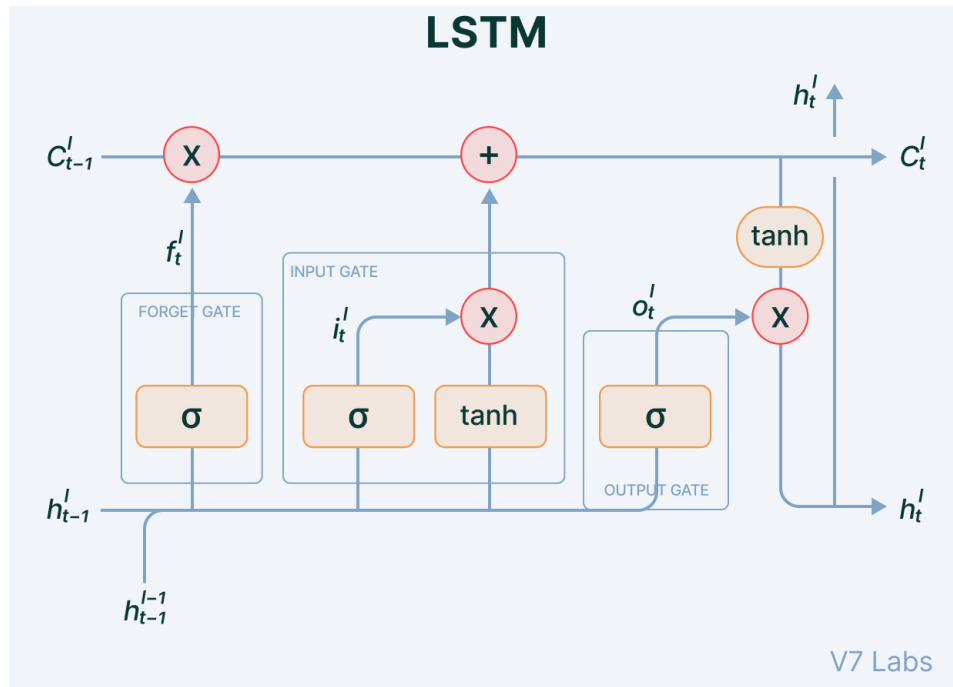
Recurrent Neural Networks (RNNs)

- It has at least one feedback loop.
- It maintain a hidden state that serves as a memory, capturing information about previous inputs in a sequence allowing them to capture temporal dependencies.
- Well-suited for speech recognition, language modeling, and time-series prediction problems.
- However, traditional RNNs have limitations in capturing long-term dependencies.



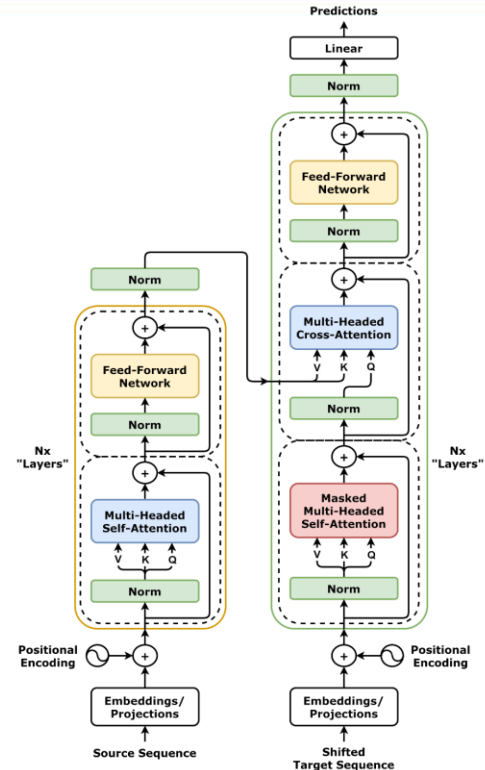
Long Short-Term Memory (LSTM)

- LSTMs are a special kind of RNN capable of learning long-term dependencies.
- **Cell State:** LSTMs have a cell state that runs through the entire sequence and can carry information across many steps.
- **Gates:** control the flow of information:
 - Input Gate: Determines which information from the current input should be updated in the cell state.
 - Forget Gate: Decides what information should be discarded from the cell state.
 - Output Gate: Controls the information that should be outputted based on the cell state.



Transformer Networks

- Transformers are the backbone of many modern NLP models.
- They process input data using self-attention, allowing for parallelization and improved handling of long-range dependencies.
 - Self-Attention Mechanism:** This mechanism computes the importance of each part of the input relative to every other part, enabling the model to weigh the significance of different words in a sentence differently.
 - Encoder-Decoder Architecture:** Consists of an encoder that processes the input sequence and a decoder that generates the output sequence. Each consists of multiple layers of self-attention and feed-forward networks.



Generative Adversarial Networks (GANs)

- GANs generate new synthetic data by training two neural networks in a competitive setting.
- They have been used to create realistic images, videos, and audio.
 - **Generator Network:** Creates fake data from random noise.
 - **Discriminator Network:** Evaluates the authenticity of the data, distinguishing between real and fake data.
 - Training Process: The generator tries to fool the discriminator by producing better fake data, while the discriminator tries to get better at detecting counterfeit data.

